



ข้อเสนอโครงร่างโครงการสหกิจศึกษา

รายวิชา 241-403 เตรียมสหกิจศึกษา

ภาคการศึกษา 1/2569

ระบบแชทบอทฝ่ายทรัพยากรบุคคลผ่านไลน์ออฟฟิเชียลแอคเคาท์

HR Chatbot System via LINE Official Account

ผู้เสนอโครงการ

นาย เจษฎา แก้วมณี

สาขาวิชาวิศวกรรมปัญญาประดิษฐ์

สถานประกอบการ

กำลังสมัคร

อาจารย์ที่ปรึกษาสหกิจ

ผศ.ดร. กฤษณ์วรา รัตนโอภาส

สาขาวิชาวิศวกรรมปัญญาประดิษฐ์ คณะวิศวกรรมศาสตร์

มหาวิทยาลัยสงขลานครินทร์

การรับรองความเป็นเอกลักษณ์

ข้าพเจ้าได้ลงนามท้ายนี้เพื่อรับรองว่าข้อเสนอโครงร่างโครงการงานสหกิจศึกษาเรื่อง “ระบบแซทเทไลต์ภาพถ่ายทรัพยากรบุคคลผ่านไลน์ออฟฟิเชียลแอคเคาท์” ข้าพเจ้าได้ดำเนินการด้วยตนเองโดยมิได้คัดลอกมาจากที่ใด หากมีส่วนหนึ่งส่วนใดที่จำเป็นต้องนำข้อความจากผลงานของบุคคลอื่นใดที่ไม่ใช่ตัวข้าพเจ้า ข้าพเจ้าได้ทำอ้างอิงถึงเอกสารเหล่านั้นไว้อย่างเหมาะสม

ข้าพเจ้าได้เข้าพบอาจารย์ที่ปรึกษาอย่างต่อเนื่องตามข้อกำหนดของรายวิชาและได้ปรับปรุงข้อเสนอโครงร่างโครงการงานสหกิจศึกษาตามที่คำแนะนำของอาจารย์ที่ปรึกษาและสถานประกอบการ

ผู้จัดทำ

เจษฎา แก้วมณี

(นาย เจษฎา แก้วมณี)

6 สิงหาคม 2569

1. ชื่อโครงการ

ภาษาไทย: ระบบแชทบอทฝ่ายทรัพยากรบุคคลผ่านไลน์ออฟฟิเชียลแอคเคาท์

ภาษาอังกฤษ: HR Chatbot System via LINE Official Account

2. ชื่อผู้เสนอโครงการ

นาย เจษฎา แก้วมณี รหัสนักศึกษา 6610110049

สาขาวิชาวิศวกรรมปัญญาประดิษฐ์

3. สถานประกอบการที่ไปปฏิบัติงานสหกิจศึกษา

กำลังสมัคร

เป็นสถานประกอบการขนาดเล็กถึงขนาดกลางที่ประสงค์นำระบบแชทบอทมาใช้สนับสนุนงานด้านทรัพยากรบุคคล เพื่อลดภาระงานเอกสารและเพิ่มความสะดวกให้แก่พนักงานภายในองค์กร

4. อาจารย์ที่ปรึกษาโครงการ

ผศ.ดร. กฤษณ์วรา รัตนโอภาส

5. ความสำคัญและที่มาของโครงการ

ปัจจุบันบริษัทขนาดเล็กและขนาดกลางในประเทศไทยจำนวนมากยังบริหารงานด้านทรัพยากรบุคคล (HR) ด้วยกระบวนการที่พึ่งพาเอกสารกระดาษหรือการสื่อสารผ่านแชทส่วนตัวระหว่างพนักงานกับฝ่าย HR โดยตรง เช่น การขอลา การสอบถามสิทธิวันลาคงเหลือ หรือการสอบถามนโยบายบริษัท ซึ่งก่อให้เกิดความล่าช้าในการดำเนินงาน ข้อมูลกระจัดกระจาย และเพิ่มภาระงานให้แก่ฝ่าย HR ที่มีบุคลากรจำกัดอยู่แล้วในขณะเดียวกันระบบ HRIS หรือซอฟต์แวร์บริหารงานบุคคลสำเร็จรูปที่มีอยู่ในท้องตลาดมักมีค่าใช้จ่ายสูงและซับซ้อนเกินความจำเป็นสำหรับองค์กรขนาดเล็ก

ด้วยเหตุนี้ โครงการนี้จึงมุ่งพัฒนาระบบแชทบอทสำหรับงาน HR โดยใช้ LINE Official Account (LINE OA) เป็นช่องทางหลัก เนื่องจากเป็นแอปพลิเคชันที่คนไทยใช้งานอยู่แล้วในชีวิตประจำวัน ทำให้พนักงานเข้าถึงบริการด้าน HR ได้ง่ายโดยไม่ต้องติดตั้งแอปพลิเคชันเพิ่มเติม และช่วยให้บริษัทขนาดเล็กถึงขนาดกลางมีเครื่องมือดิจิทัลที่ใช้งานได้จริงในต้นทุนที่เหมาะสม

6. วัตถุประสงค์

- 6.1. เพื่อวิเคราะห์ ออกแบบ และพัฒนาระบบแชทบอทสำหรับงานด้านทรัพยากรบุคคลบนแพลตฟอร์ม LINE Official Account ที่สอดคล้องกับพฤติกรรมการใช้งานของพนักงาน และตอบโต้ภัยกระบวนการทำงานจริงของบริษัทขนาดเล็กและขนาดกลาง
- 6.2. เพื่อพัฒนาเว็บแอปพลิเคชันส่วนจัดการ (Admin Dashboard) สำหรับให้เจ้าหน้าที่ HR ใช้เป็นศูนย์กลางในการบริหารจัดการข้อมูลประวัติพนักงาน พิจารณานุมัติคำขอลา และเผยแพร่ประกาศข่าวสารขององค์กรได้อย่างมีประสิทธิภาพ
- 6.3. เพื่อออกแบบโครงสร้างสถาปัตยกรรมฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ที่มีความยืดหยุ่น รองรับการขยายตัวของข้อมูล (Scalability) และเตรียมความพร้อมสำหรับการเชื่อมต่อกับระบบ HRIS หรือระบบจ่ายเงินเดือน (Payroll) เดิมของบริษัทในอนาคต
- 6.4. เพื่อประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ (AI) ในการยกระดับความสามารถของแชทบอทให้สามารถตอบคำถามด้านทรัพยากรบุคคลที่เป็นภาษาธรรมชาติได้อย่างแม่นยำ

7. ขอบเขตของโครงการ

7.1. ด้านการพัฒนาระบบหลังบ้าน (Backend):

พัฒนา Backend API ด้วยเว็บเฟรมเวิร์ก FastAPI (Python) เพื่อทำหน้าที่รับส่งข้อมูลและเชื่อมต่อกับ LINE Messaging API ผ่านระบบ Webhook รวมถึงจัดการตรรกะทางธุรกิจ (Business Logic) ทั้งหมดของระบบ

7.2. ด้านการพัฒนาระบบหน้าบ้าน (Frontend):

พัฒนา Admin Dashboard ด้วยเฟรมเวิร์ก Next.js (React) สำหรับให้ทีม HR ใช้ในการบริหารจัดการระบบแบบรวมศูนย์

7.3. ด้านการจัดการฐานข้อมูล (Database):

ออกแบบและพัฒนาระบบฐานข้อมูลด้วยระบบ PostgreSQL โดยครอบคลุมการจัดเก็บข้อมูลเอนทิตีที่สำคัญ ได้แก่ ข้อมูลพนักงาน ประวัติการขอลา ฐานข้อมูลคำถามที่พบบ่อย (FAQ) และระบบประกาศข่าวสารบริษัท โดยตารางข้อมูลพนักงานจะมีคอลัมน์ระบุสิทธิ์การใช้งาน (Role) เพื่อแยกระดับการเข้าถึง เช่น ผู้ดูแลระบบ เจ้าหน้าที่ HR และพนักงานทั่วไป พร้อมทั้งมีตารางสำหรับกระบวนกร

ยืนยันตัวตนและผูกบัญชี LINE เข้ากับข้อมูลพนักงานในระบบ เพื่อให้ Backend สามารถคัดกรองสิทธิ์การเข้าถึง Admin Dashboard ได้อย่างถูกต้อง ส่วนตารางคำขอลาจะมีคอลัมน์สำหรับจัดเก็บอ้างอิงไฟล์แนบ เช่น ใบรับรองแพทย์หรือหลักฐานประกอบการลา เพื่อรองรับกรณีที่พนักงานต้องส่งเอกสารประกอบคำขอผ่านระบบ นอกจากนี้ยังติดตั้งส่วนขยาย pgvector บน PostgreSQL เพื่อทำหน้าที่เป็น Vector Database สำหรับจัดเก็บเวกเตอร์ (Embedding) ของเอกสารความรู้บริษัท เช่น คู่มือพนักงานและนโยบาย HR ที่ผ่านการตัดแบ่งเป็น Chunk ไว้แล้ว เพื่อใช้ในการค้นคืนข้อมูลประกอบการตอบคำถามด้วยเทคนิค RAG

7.4. ด้านฟีเจอร์การใช้งานหลัก (Core Features):

ครอบคลุมระบบการยื่นขอลาและตรวจสอบวันลาคงเหลือแบบอัตโนมัติ, การตอบคำถามที่พบบ่อย (FAQ), การแจ้งเตือนประกาศบริษัท และระบบการอนุมัติหรือปฏิเสธคำขอลาผ่าน LINE

7.5. พัฒนาโมดูลตอบคำถามอัตโนมัติด้วย Large Language Model (LLM)

เช่น Claude API

เพื่อรองรับคำถามของพนักงานที่เป็นภาษาธรรมชาติและอยู่นอกเหนือจากชุดคำถามที่พบบ่อย (FAQ) ที่กำหนดไว้ล่วงหน้า โดยระบบจะมีกลไก Semantic Cache คั่นกลางก่อนเรียก External LLM เพื่อลดจำนวนครั้งในการเรียก API สำหรับคำถามที่มีความหมายใกล้เคียงกัน ซึ่งช่วยลดทั้งค่าใช้จ่าย Token และเวลาในการตอบสนอง (Latency) และก่อนส่งข้อมูลจาก Vector Database ไปยัง External LLM ระบบจะมีขั้นตอน Data Masking เพื่อปกปิดข้อมูลส่วนบุคคล (PII) ของพนักงานตามหลักการคุ้มครองข้อมูลส่วนบุคคล (PDPA) ก่อนส่งออกไปประมวลผลภายนอก

7.6. ประยุกต์ใช้เทคนิค Retrieval-Augmented Generation (RAG)

ในการค้นคืนข้อมูลจากฐานความรู้เฉพาะของบริษัท (เช่น คู่มือพนักงาน, นโยบาย HR) เพื่อส่งให้ LLM ใช้ประกอบการสร้างคำตอบ ซึ่งจะช่วยลดปัญหาการสร้างข้อมูลที่ผิดพลาด (Hallucination) ของโมเดล

7.7. ติดตั้งและใช้งาน Langfuse

เป็นเครื่องมือด้าน LLM Observability สำหรับติดตาม ตรวจสอบ และประเมินคุณภาพการทำงานของระบบ AI โดยสามารถบันทึก Trace ของบทสนทนา ประเมินความแม่นยำ และตรวจสอบปริมาณการใช้ Token รวมถึงค่าใช้จ่าย/เวลาในการตอบสนอง (Latency)

7.8. ขอบเขตของโครงการไม่รวมการฝึกฝนโมเดล (Fine-tuning) ขึ้นเอง โดยจะใช้งานผ่าน API ของผู้ให้บริการ LLM เท่านั้น

7.9. โครงการนี้ไม่มีการเชื่อมต่อกับระบบบริหารเงินเดือน (Payroll System) จริงของบริษัท เนื่องจากมีความซับซ้อนและอยู่นอกเหนือกรอบระยะเวลาการปฏิบัติงานสหกิจศึกษา

8. ประโยชน์ที่คาดว่าจะได้รับ

8.1. ประโยชน์ต่อสถานประกอบการ:

ช่วยลดภาระงานด้านเอกสาร การดำเนินการที่ซ้ำซ้อน และลดเวลาที่เจ้าหน้าที่ HR ต้องใช้ในการตอบคำถามพื้นฐาน ทำให้ฝ่ายทรัพยากรบุคคลสามารถบริหารจัดการข้อมูลได้อย่างเป็นระบบและรวดเร็วขึ้น

8.2. ประโยชน์ต่อพนักงาน:

พนักงานได้รับประสบการณ์ที่ดีขึ้นในการเข้าถึงบริการด้าน HR โดยสามารถสอบถามข้อมูล ทำเรื่องขอลา หรือติดตามข่าวสารขององค์กรได้ด้วยตนเองอย่างสะดวกรวดเร็วผ่านแอปพลิเคชันที่ใช้งานอยู่เป็นประจำ

8.3. ประโยชน์ในเชิงธุรกิจและการต่อยอด:

โครงการนี้จะก่อให้เกิดต้นแบบระบบ (Prototype) ที่มีความสมบูรณ์ในระดับที่สามารถนำไปทดสอบใช้งานจริง และมีศักยภาพในการนำไปต่อยอดพัฒนาเป็นผลิตภัณฑ์ซอฟต์แวร์ในรูปแบบ Software as a Service (SaaS) เพื่อเจาะตลาดกลุ่ม SME ได้ในอนาคต

8.4. ประโยชน์ต่อผู้จัดทำโครงการ:

ผู้จัดทำได้ประยุกต์ใช้ความรู้จากสาขาวิชาวิศวกรรมปัญญาประดิษฐ์มาแก้ปัญหาที่เกิดขึ้นจริงในภาคอุตสาหกรรม ได้ฝึกฝนทักษะการพัฒนาระบบซอฟต์แวร์แบบ Full-stack การออกแบบสถาปัตยกรรมระบบ การบูรณาการ AI เข้ากับแอปพลิเคชัน (AI Integration) ตลอดจนเรียนรู้การทำงานร่วมกับทีมงานในสถานประกอบการจริง ซึ่งเป็นการเตรียมความพร้อมก่อนเข้าสู่โลกของการทำงานได้อย่างสมบูรณ์

9. ทฤษฎีและหลักการ

9.1 LINE Messaging API

LINE Messaging API เป็นชุดเครื่องมือและอินเทอร์เฟซการเขียนโปรแกรมที่ LINE Corporation จัดเตรียมไว้สำหรับนักพัฒนา เพื่อใช้ในการสร้างแชทบอทที่สามารถสื่อสารโต้ตอบกับผู้ใช้งานผ่านหน้าต่างแชทของ LINE Official Account ได้อย่างมีประสิทธิภาพและเป็นธรรมชาติ การทำงานของระบบอาศัยกลไก Webhook เป็นหลัก โดยเมื่อผู้ใช้งานมีปฏิสัมพันธ์กับระบบ (เช่น การส่งข้อความ, การกดปุ่มบน Rich Menu, หรือการส่งรูปภาพ) เซิร์ฟเวอร์ของ LINE จะส่ง Event Data ในรูปแบบ JSON มายังระบบเซิร์ฟเวอร์ของผู้พัฒนาแบบเรียลไทม์ จากนั้นระบบสามารถประมวลผลและส่งข้อความตอบกลับไปยังผู้ใช้งานผ่าน Reply API หรือสามารถส่งข้อความแจ้งเตือนเชิงรุกผ่าน Push API ได้ นอกจากนี้ยังรองรับการแสดงผลข้อความที่มีโครงสร้างซับซ้อน (Flex Message) ซึ่งช่วยเพิ่มประสิทธิภาพในการสื่อสารข้อมูลทางด้าน HR เช่น การแสดงตารางวันลาหรือสลิปเงินเดือนให้มีความสวยงามและอ่านง่าย [1]

การใช้งานในโครงการนี้: ระบบตั้งค่า Webhook Endpoint ไว้ที่ฝั่ง Backend (FastAPI) เพื่อรับ Event เมื่อพนักงานพิมพ์ข้อความสอบถาม เช่น การขอลาหรือตรวจสอบวันลาคงเหลือ หรือกดปุ่มบน Rich Menu จากนั้นระบบประมวลผลและตอบกลับผ่าน Reply API พร้อมใช้ Flex Message แสดงข้อมูล เช่น ตารางสรุปวันลาคงเหลือ ให้อยู่ในรูปแบบที่อ่านง่ายและเป็นระเบียบ

9.2 สถาปัตยกรรมเว็บแบบ Client-Server และ REST API

โครงการนี้ประยุกต์ใช้สถาปัตยกรรมระบบแบบ Client-Server ร่วมกับหลักการ Separation of Concerns โดยแยกส่วนการทำงานระหว่างฝั่งหน้าบ้าน (Frontend) และระบบหลังบ้าน (Backend) อย่างชัดเจน

- **Frontend:** พัฒนาด้วย Next.js ซึ่งเป็นเฟรมเวิร์กที่ต่อยอดมาจาก React มีจุดเด่นในการรองรับการเรนเดอร์ผลฝั่งเซิร์ฟเวอร์ (Server-side Rendering: SSR) ช่วยให้การแสดงผล Admin Dashboard รวดเร็ว ปลอดภัย และจัดการสถานะของแอปพลิเคชัน (State Management) ได้อย่างมีประสิทธิภาพ [2] โดยพัฒนาโค้ดฝั่ง Frontend ด้วยภาษา TypeScript [7] เพื่อเพิ่มความปลอดภัยของชนิดข้อมูล (Type Safety) และบริหารจัดการแพ็คเกจรวมถึงรันไทม์ด้วย Bun [8] ซึ่งมีความเร็วในการติดตั้งและประมวลผลสูงกว่าเครื่องมือดั้งเดิม

- **Backend:** พัฒนาด้วย FastAPI [3] ซึ่งเป็น Web Framework ของภาษา Python ที่ออกแบบมาสำหรับการสร้าง API โดยเฉพาะ มีจุดเด่นด้านความเร็วในการประมวลผลระดับสูง (High Performance) รองรับการทำงานแบบ Asynchronous และมีการตรวจสอบโครงสร้างข้อมูล (Data Validation) อย่างเข้มงวดด้วย Pydantic [10] พร้อมทั้งบริหารจัดการ Dependency และสภาพแวดล้อมของโปรเจกต์ด้วย uv [9] ซึ่งเป็นเครื่องมือจัดการแพ็คเกจ Python ที่มีความเร็วในการติดตั้งสูง ทั้งสองฝั่งนี้จะเชื่อมต่อและแลกเปลี่ยนข้อมูลกันผ่านสถาปัตยกรรม REST API (Representational State Transfer) โดยใช้รูปแบบโครงสร้างข้อมูลแบบ JSON (JavaScript Object Notation) ซึ่งเป็นมาตรฐานสากลที่ระบบต่างแพลตฟอร์มสามารถทำความเข้าใจร่วมกันได้

9.3 ระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (RDBMS) และแผนภาพ ER

เนื่องจากข้อมูลด้านทรัพยากรบุคคลเป็นข้อมูลที่มีโครงสร้างชัดเจนและต้องการความถูกต้องแม่นยำสูง โครงการนี้จึงเลือกใช้ PostgreSQL ซึ่งเป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database Management System) แบบ Open Source ที่มีประสิทธิภาพสูงและรองรับมาตรฐาน ACID (Atomicity, Consistency, Isolation, Durability) เพื่อรับประกันความสมบูรณ์ของข้อมูล [4] ก่อนกระบวนการพัฒนาระบบจริง จะต้องมีการออกแบบสถาปัตยกรรมข้อมูลด้วยการเขียนแผนภาพความสัมพันธ์ระหว่างเอนทิตี (Entity-Relationship Diagram: ERD) เพื่อกำหนดตารางข้อมูล (Tables) คุณลักษณะของข้อมูล (Attributes) และสร้างความสัมพันธ์ (Relationships) ระหว่างตารางต่างๆ ผ่านคีย์หลัก (Primary Key) และคีย์นอก (Foreign Key) การออกแบบ ERD ที่ดีตามหลักการ Normalization จะช่วยลดความซ้ำซ้อนของข้อมูล (Data Redundancy) และป้องกันความผิดปกติในการจัดการข้อมูล (Data Anomalies)

การใช้งานในโครงการนี้: ฐานข้อมูลของระบบประกอบด้วยตารางหลัก ได้แก่ employees (รวมคอลัมน์ role สำหรับแยกสิทธิ์ผู้ใช้งาน), leave_requests (รวมคอลัมน์อ้างอิงไฟล์แนบสำหรับหลักฐานประกอบการลา), leave_balances, faqs และ announcements ที่เชื่อมโยงกันผ่าน Primary Key และ Foreign Key ตามแผนภาพ ER ที่ออกแบบไว้ (ดังแสดงในรูปที่ 1) เพื่อรองรับการขยายเชื่อมต่อกับระบบ HRIS หรือระบบเงินเดือนของบริษัทในอนาคต ทั้งนี้ระบบจะไม่จัดเก็บบทสนทนาฉบับเต็มซ้ำซ้อนใน PostgreSQL เนื่องจากการบันทึก Trace และประวัติการสนทนาของโมดูล AI ถูกจัดการโดย

Langfuse อยู่แล้ว โดยฐานข้อมูลเชิงสัมพันธ์จะเก็บเฉพาะข้อมูลธุรกรรมที่จำเป็น เช่น สถานะคำขอลา เพื่อลดความซ้ำซ้อนของข้อมูล

9.4 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) เป็นการผสมผสานความสามารถของการค้นคืนข้อมูล (Information Retrieval) เข้ากับการสร้างข้อความของ LLM โดยเมื่อพนักงานสอบถามข้อมูล ระบบจะทำการค้นหาเนื้อหาที่เกี่ยวข้องที่สุดจากฐานความรู้เฉพาะของบริษัท (เช่น คู่มือพนักงาน, กฎระเบียบ HR) ก่อน แล้วจึงแนบข้อมูลเหล่านั้นไปพร้อมกับ (Prompt) เพื่อให้ LLM ใช้เป็นบริบทอ้างอิงในการสังเคราะห์คำตอบ วิธีนี้ช่วยลดข้อจำกัดของระบบแชทบอทแบบ Rule-based ทั่วไป และทำให้ผู้ใช้ได้รับคำตอบที่มีความถูกต้อง น่าเชื่อถือ และอ้างอิงจากนโยบายจริงของบริษัทได้อย่างแม่นยำ

กระบวนการทำงานของ RAG ประกอบด้วยขั้นตอนหลัก 4 ขั้นตอน เริ่มจาก (1) TextSplitter ทำหน้าที่แบ่งเอกสารต้นฉบับที่มีขนาดยาว เช่น คู่มือพนักงานหรือกฎระเบียบ HR ออกเป็นส่วนย่อยที่เรียกว่า Chunk เพื่อให้ขนาดของข้อมูลเหมาะสมกับการประมวลผลและการค้นคืน (2) นำ Chunk แต่ละส่วนไปแปลงเป็นเวกเตอร์เชิงตัวเลขด้วยกระบวนการ Embedding ซึ่งแทนความหมายเชิงบริบทของข้อความในรูปแบบที่คอมพิวเตอร์เปรียบเทียบความคล้ายคลึงกันได้ (3) จัดเก็บเวกเตอร์เหล่านั้นไว้ใน Vector Database ซึ่งเป็นฐานข้อมูลที่ออกแบบมาสำหรับค้นหาความคล้ายคลึงเชิงความหมาย (Semantic Similarity Search) โดยเฉพาะ และ (4) เมื่อพนักงานตั้งคำถาม ระบบจะใช้ Retriever ค้นหา Chunk ที่มีความหมายใกล้เคียงกับคำถามมากที่สุดจาก Vector Database แล้วส่งต่อไปให้ LLM ใช้เป็นบริบทประกอบการตอบกลับ ทั้งนี้โครงงานนี้เลือกใช้ LangChain [12] ซึ่งเป็นเฟรมเวิร์ก Open Source สำหรับพัฒนาแอปพลิเคชันที่ขับเคลื่อนด้วย LLM มาช่วยเชื่อมประสานทั้ง 4 ขั้นตอนของ RAG เข้าด้วยกันในรูปแบบ Pipeline เดียว ลดความซับซ้อนในการเขียนโค้ดเชื่อมต่อระหว่างเครื่องมือแต่ละส่วนด้วยตนเอง

การใช้งานในโครงงานนี้: ระบบใช้ LangChain [12] เป็นเฟรมเวิร์กหลักในการเชื่อมประสาน Pipeline ของ RAG ร่วมกับการเรียกใช้ LLM ผ่าน API (เช่น Claude API) โดยไม่มีการฝึกฝนโมเดลขึ้นเอง เอกสารความรู้ของบริษัท เช่น คู่มือพนักงานและนโยบายการลา จะถูกแบ่งเป็น Chunk ด้วย TextSplitter ของ LangChain แล้วแปลงเป็นเวกเตอร์ด้วย Embedding Model ก่อนจัดเก็บลงใน Vector Database (เช่น ส่วนขยาย pgvector [11] บน PostgreSQL ที่ใช้งานอยู่แล้ว) เมื่อพนักงานสอบถามคำถามที่อยู่นอกเหนือจาก FAQ

ที่กำหนดไว้ล่วงหน้า Retriever ของ LangChain จะค้นหา Chunk ที่เกี่ยวข้องที่สุดจาก Vector Database แล้วส่งเป็นบริบทประกอบการตอบกลับผ่านเทคนิค RAG เพื่อให้คำตอบมีความถูกต้องและสอดคล้องกับนโยบายจริงของบริษัท

9.5. LLM Observability และเครื่องมือ Langfuse

เนื่องจากการประมวลผลของโมเดลภาษาขนาดใหญ่ (LLM) มีลักษณะไม่แน่นอนตายตัว (Non-deterministic) การพัฒนาระบบซอฟต์แวร์ที่มี AI เป็นส่วนประกอบหลักจึงจำเป็นต้องมีกระบวนการสังเกตการณ์ ตรวจสอบ และติดตามผล (LLM Observability) อย่างใกล้ชิด โครงการนี้เลือกใช้ **Langfuse** [5] ซึ่งเป็นเครื่องมือ Open Source สำหรับงาน LLM Engineering โดยเฉพาะ เข้ามาทำหน้าที่ติดตามการทำงานเชิงลึก เครื่องมือนี้มีความสามารถในการบันทึกร่องรอย (Trace) ของทุกๆ การสนทนา เก็บข้อมูล Input, Output และ Prompt ที่ถูกใช้งานจริง (Prompt Management) เพื่อนำมาประเมินคุณภาพความแม่นยำของคำตอบ (Evaluation) นอกจากนี้ยังสามารถติดตามและคำนวณปริมาณการใช้ Token, ค่าใช้จ่ายที่เกิดขึ้น (Cost Tracking), และระยะเวลาในการตอบสนองของโมเดล (Latency) ข้อมูลเชิงลึกเหล่านี้มีความสำคัญอย่างยิ่งต่อผู้พัฒนาในการนำไปใช้ปรับปรุงคุณภาพการให้บริการของแชทบอทให้ดียิ่งขึ้นอย่างต่อเนื่องแบบ Data-driven

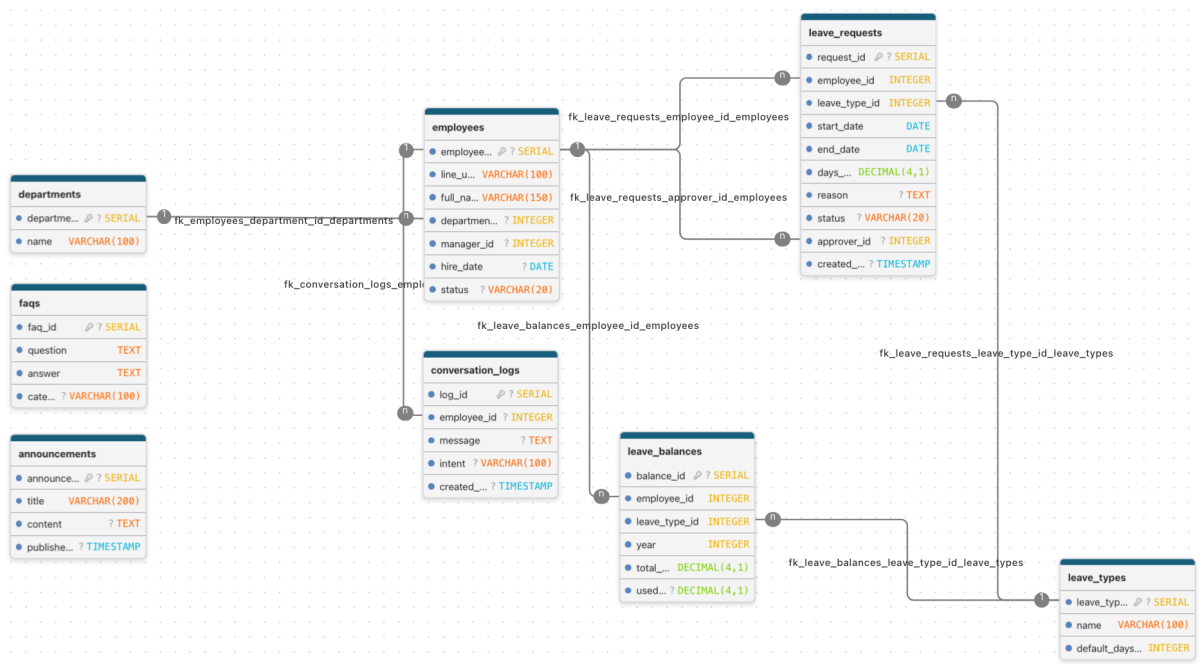
การใช้งานในโครงการนี้: ทุกการสนทนาที่ผ่านโมดูล AI ของระบบจะถูกส่ง Trace ไปบันทึกที่ Langfuse โดยอัตโนมัติ ทำให้ผู้พัฒนาตรวจสอบคุณภาพคำตอบ ปริมาณการใช้ Token และค่าใช้จ่ายได้แบบเรียลไทม์ ทั้งในระหว่างการพัฒนาและหลังจากนำระบบไปใช้งานจริงกับพนักงาน

10. ขั้นตอนและวิธีดำเนินการ

10.1 การศึกษาและวิเคราะห์ความต้องการ (Requirement Analysis): ศึกษาและรวบรวมข้อมูลกระบวนการทำงานเดิม (As-Is Process) ของฝ่ายทรัพยากรบุคคล (HR) ภายในสถานประกอบการ เพื่อวิเคราะห์ปัญหาและความต้องการของผู้ใช้งาน (User Requirements) จากนั้นจึงนำมากำหนดขอบเขตและฟังก์ชันการทำงานของระบบแชทบอทให้ตอบโจทย์การแก้ปัญหาอย่างตรงจุด

10.2 การออกแบบสถาปัตยกรรมระบบและฐานข้อมูล (System & Database Design): ออกแบบภาพรวมของสถาปัตยกรรมระบบ (System Architecture) ที่แสดงการเชื่อมโยงระหว่าง Frontend, Backend และ External APIs ตลอดจนออกแบบโครงสร้างฐานข้อมูลเชิงสัมพันธ์ โดย

จัดทำแผนภาพความสัมพันธ์ระหว่างเอนทิตี (Entity-Relationship Diagram: ERD) เพื่อใช้เป็นแม่แบบในการสร้างฐานข้อมูล



รูปที่ 1 แผนภาพความสัมพันธ์ระหว่างเอนทิตี (ER Diagram) ของฐานข้อมูลระบบ

10.3. การพัฒนาระบบส่วนหลังบ้านและการเชื่อมต่อ API (Backend Development & Integration): พัฒนา Backend API ด้วยเว็บเฟรมเวิร์ก FastAPI (Python) เพื่อจัดการตรรกะทางธุรกิจ (Business Logic) ของระบบ พร้อมทั้งตั้งค่า Webhook เพื่อเชื่อมต่อและรับส่งข้อมูลข้อความแบบเรียลไทม์กับ LINE Messaging API



รูปที่ 2 FastAPI

10.4 การพัฒนาระบบส่วนหน้าบ้าน (Frontend Development): พัฒนาเว็บแอปพลิเคชันส่วนจัดการ (Admin Dashboard) ด้วยเฟรมเวิร์ก Next.js เพื่อเป็นส่วนต่อประสานกับผู้ใช้งาน (User Interface) ให้เจ้าหน้าที่ HR สามารถเข้ามาบริหารจัดการข้อมูลพนักงาน อนุมัติการลา และตรวจสอบสถิติต่าง ๆ ได้อย่างสะดวก



รูปที่ 3 Next.js

10.5. การพัฒนาโมดูลปัญญาประดิษฐ์ (AI Module Development): บูรณาการโมเดลภาษาขนาดใหญ่ (LLM) เข้ากับระบบเพื่อรองรับการตอบคำถามอัตโนมัติ โดยประยุกต์ใช้เทคนิค Retrieval-Augmented Generation (RAG) ในการดึงข้อมูลจากฐานความรู้ของบริษัทมาประกอบการตอบคำถาม โดยใช้เฟรมเวิร์ก LangChain เชื่อมประสาน Pipeline ตั้งแต่การแบ่งเอกสารด้วย TextSplitter การแปลงข้อมูลเป็นเวกเตอร์ (Embedding) การจัดเก็บลงใน Vector Database ด้วยส่วนขยาย pgvector บน PostgreSQL ไปจนถึงการค้นคืนข้อมูลด้วย Retriever พร้อมทั้งเชื่อมต่อระบบเข้ากับเครื่องมือ Langfuse เพื่อสังเกตการณ์ (Observability) ติดตามการใช้งาน และประเมินผลความถูกต้องของคำตอบ AI

10.6. การทดสอบระบบ (System Testing & Evaluation): นำระบบต้นแบบ (Prototype) ไปทดสอบการใช้งานจริง (User Acceptance Testing: UAT) ร่วมกับพนักงานและเจ้าหน้าที่ HR ภายในสถานประกอบการ เพื่อตรวจสอบความถูกต้องของฟังก์ชันการทำงาน ค้นหาข้อผิดพลาด (Bugs) และเก็บรวบรวมข้อเสนอแนะ (Feedback) จากผู้ใช้งานจริง

10.7. การปรับปรุงระบบและจัดทำเอกสาร (System Refinement & Documentation): นำข้อเสนอแนะและผลการทดสอบที่ได้มาปรับปรุงแก้ไขระบบให้มีความสมบูรณ์และเสถียรภาพมากยิ่งขึ้น พร้อมทั้งจัดทำคู่มือการใช้งานระบบและเอกสารรายงานโครงการงานสหกิจศึกษาฉบับสมบูรณ์เพื่อนำเสนอต่อสถานประกอบการและมหาวิทยาลัย

11. แผนการดำเนินการ

กิจกรรม	เดือนที่ 1	เดือนที่ 2	เดือนที่ 3	เดือนที่ 4
วิเคราะห์ความต้องการและออกแบบระบบ				
ออกแบบฐานข้อมูลและสถาปัตยกรรม				
พัฒนา Backend API (FastAPI)				
พัฒนา Admin Dashboard (Next.js)				
พัฒนาโมดูล AI (RAG + Langfuse)				
ทดสอบระบบและเก็บผลตอบรับ				
ปรับปรุงระบบและจัดทำเอกสาร				

ตารางที่ 1 แผนการดำเนินงานโครงการตลอดระยะเวลาสหกิจศึกษา

12. เอกสารอ้างอิง

- [1] LINE Corporation, "LINE Messaging API Documentation," [Online]. Available: <https://developers.line.biz/en/docs/messaging-api/>. [Accessed: 16-Jul-2026].
- [2] Vercel Inc., "Next.js Documentation," [Online]. Available: <https://nextjs.org/docs>. [Accessed: 16-Jul-2026].
- [3] FastAPI, "FastAPI Documentation," [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: 16-Jul-2026].
- [4] PostgreSQL Global Development Group, "PostgreSQL Documentation," [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed: 16-Jul-2026].
- [5] Langfuse, "Langfuse Documentation," [Online]. Available: <https://langfuse.com/docs>. [Accessed: 16-Jul-2026].
- [6] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [7] Microsoft, "TypeScript Documentation," [Online]. Available: <https://www.typescriptlang.org/docs/>. [Accessed: 16-Jul-2026].
- [8] Oven Technologies, Inc., "Bun Documentation," [Online]. Available: <https://bun.sh/docs>. [Accessed: 16-Jul-2026].
- [9] Astral Software Inc., "uv Documentation," [Online]. Available: <https://docs.astral.sh/uv/>. [Accessed: 16-Jul-2026].
- [10] Pydantic Services Inc., "Pydantic Documentation," [Online]. Available: <https://docs.pydantic.dev/>. [Accessed: 16-Jul-2026].
- [11] pgvector Contributors, "pgvector: Open-source vector similarity search for Postgres," [Online]. Available: <https://github.com/pgvector/pgvector>. [Accessed: 16-Jul-2026].
- [12] LangChain, "LangChain Documentation," [Online]. Available: <https://python.langchain.com/docs/>. [Accessed: 16-Jul-2026].